```
decl.c:2:1: warning: symbol 'foo' was not declared. Should it
be static?
```

The *'-Wdecl'* option is enabled by default, and detects any non-static variables or functions. You can disable it with the *'-Wno-decl'* option. To fix the warning, the function *foo()* should be declared static. A similar output was observed when Sparse was run on Linux 3.10.9 kernel sources:

```
arch/x86/crypto/fpu.c:153:12: warning: symbol 'crypto_fpu_
init' was not declared. Should it be static?
```

While the C99 standard allows declarations after a statement, the C89 standard does not permit it. The following *decl-after.c* example includes a declaration after an assignment statement:

```
int
main (void)
{
  int x;

  x = 3;

  int y;

  return 0;
}
```

When using the C89 standard with the *'-ansi'* or *'-std=c89'* option, Sparse emits a warning, as shown below:

```
$ sparse -I/usr/include/linux \
         -I/usr/lib/gcc/x86_64-redhat-linux/4.7.2/include \
         -I/usr/include -ansi decl-after.c

decl-after.c:8:3: warning: mixing declarations and code
```

This Sparse command line step can be automated with a *Makefile*:

```
TARGET = decl-after

SPARSE_INCLUDE = -I/usr/include/linux \
      -I/usr/lib/gcc/x86_64-redhat-linux/4.7.2/include \
      -I/usr/include

SPARSE_OPTIONS = -ansi

all:
    sparse $(SPARSE_INCLUDE) $(SPARSE_OPTIONS) $(TARGET).c

clean:
    rm -f $(TARGET) *~ a.out
```

If a void expression is returned by a function whose return type is void, Sparse issues a warning. This option needs to be explicitly specified with *'-Wreturn-void'*. For example:

```
static void
foo (int y)
{
  int x = 1;

  x = x + y;
}

static void
fd (void)
{
  return foo(3);
}

int
main (void)
{
  fd();

  return 0;
}
```

Executing the above code with Sparse results in the following output:

```
$ sparse -I/usr/include/linux \
     -I/usr/lib/gcc/x86_64-redhat-linux/4.7.2/include \
     -I/usr/include -Wreturn-void void.c

void.c:12:3: warning: returning void-valued expression
```

The *'-Wcast-truncate'* option warns about truncation of bits during casting of constants. This is enabled by default. An 8-bit character is assigned more than it can hold in the following code:

```
int
main (void)
{
  char i = 0xFFFF;

  return 0;
}
```

Sparse warns of truncation for the above code:

```
$ sparse -I/usr/include/linux \
```